Software Requirements Specification

for

Code Plagiarism Detector

Version 1.1

Pantheon

February 10, 2026

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

Revision History iii
1. Introduction 1
1.1 Document Purpose 1
1.2 Document Conventions 1
1.3 Project Scope 1
1.4 References 1
2. Overall Description 1
2.1 Product Perspective 2
2.2 User Classes and Characteristics 2
2.3 Design and Implementation Constraints 2
2.4 Assumptions and Dependencies 3
3. System Features 4
3.1 User Authentication and Authorization 4
3.2 Course and Assignment Management 4
3.3 Code Submission 5
3.4 Repository Management 5
3.5 Similarity Analysis 5
4. Data Requirements 6
4.1 Logical Data Model 6
4.2 Data Dictionary 7
4.3 Reports 7
4.4 Data Acquisition, Integrity, Retention, and Disposal 8
4.4.1 Data Acquisition 8
4.4.2 Data Integrity and Protection 8
4.4.3 Data Retention 8
4.4.4 Data Disposal 8
4.4.5 Privacy and Anonymization 9
5. External Interface Requirements 9
5.1 User Interfaces 9
5.2 Software Interfaces 9
5.3 Communications Interfaces 10
6. Quality Attributes 10
6.1 Usability 10
6.2 Performance 10
6.3 Security 10
6.4 Privacy/Confidentiality 11
6.5 Reliability/Availability 11
6.6 Maintainability/Modifiability 11
6.7 Portability/Deployment 11
7. Glossary 11
8. Host Software Requirements and Viability 12
8.1 Purpose 12
8.2 Client Environment Requirements 12
8.3 Server Operating System Requirements 12
8.4 Application Runtime Environment 12
8.5 Requirements for Database Management System (DBMS) 13
8.6 File Storage Requirements 13
8.7 Network and Communication Requirements 13
8.8 Hardware Requirements 14
8.9 Viability and Feasibility Assessment 14
9. Analysis Models 14

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

Revision History

Name	Date	Reason For Changes	Version and Document
Pantheon	Jan 30, 2026	Initial document startup	1.0
Pantheon	February 10, 2026	Document Complete for Stage 1	1.1

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

1. Introduction

This document presents the Software Requirements Specification (SRS) for the Pantheon project. The purpose of this document is to outline the requirements and design considerations for a web-based system that supports academic integrity by analyzing student programming submissions for code similarity. The SRS serves as a foundation for the design and development of the system and helps ensure that all group members and stakeholders share a common understanding of what the system is expected to do.

1.1 Document Purpose

The purpose of this document is to define the functional and non-functional requirements for the Pantheon system. It describes the expected behavior of the system, its intended users, and the constraints under which it will operate. This document is intended for the project development team, instructors, and evaluators, and will be used as a reference throughout the remainder of the project.

1.2 Document Conventions

This document follows the standard Software Requirements Specification format as provided in the course template. Diagrams, figures, and tables are labeled and referenced where appropriate.

1.3 Project Scope

The Pantheon system is a web-based application that allows students to submit programming assignments electronically and enables instructors to analyze submissions for code similarity. Submitted code is stored in a repository and analyzed against other submissions, previous offerings, or instructor-defined repositories to identify potential academic integrity concerns.

The system provides instructors with a user-friendly interface to view similarity reports and compare code submissions while ensuring student confidentiality through anonymization. The minimum supported programming languages for analysis are C, C++, and Java. The system is designed to operate over the internet and may be hosted on any suitable server environment.

1.4 References

- GitHub repository for the Pantheon project
- Brock University Academic Integrity Policy
- Software Requirements Specification Template (Karl Wiegers)
- Bockus, D. COSC 4P01 Lecture Slides, Weeks 1–12, Brock University, 2025.
- Sommerville, I. Software Engineering (10th Edition). Pearson Education, 2016. ISBN: 978-0-13-394303-0.

2. Overall Description

This section provides a high-level overview of a web-based code submission and similarity analysis system used to support academic integrity review. Students submit programming assignments through an online portal. Submissions are stored in a primary repository (e.g., per course offering) and can be analyzed against

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

other submissions in the same offering as well as instructor-selected secondary repositories (e.g., prior terms or curated archives). The system generates similarity results and provides instructors with a web interface to review ranked matches and perform side-by-side comparisons. To comply with privacy and confidentiality requirements, the system anonymizes submissions and restricts identity disclosure to authorized roles with auditable access.

2.1 Product Perspective

This product is a standalone system designed to support academic integrity checks for programming assignments. It integrates into the academic workflow by accepting student submissions and providing similarity analysis outputs for instructional staff. At a high level, the system consists of:

- Web client interfaces (student and instructor portals)
- Backend services (submission handling, repository management, analysis job orchestration)
- Similarity comparison engine
- Persistent storage (raw submissions, normalized representations, reports)
- Database (users/roles if used, submission metadata, analysis results, audit logs)

2.2 User Classes and Characteristics

Student (primary)

- Submits assignments/projects via upload (single file or archive, as defined by course policy)
- Expects fast confirmation, clear validation errors, and confidentiality protection
- Does not access similarity results or other students' submissions

Instructor (favored user class)

- Configures repositories (selects primary and optional secondary repositories)
- Initiates analysis runs, reviews similarity indices, and performs detailed pairwise comparisons
- Expects efficient triage (top matches, filters, thresholds) and easy navigation across files/functions

Teaching Assistant (optional)

- May assist with review depending on instructor permissions
- Typically needs read-only access to analysis results and comparison tools

System Administrator (project/hosting role)

- Manages deployment, storage limits, backups, access control configuration, and monitoring
- Does not perform academic review, but can manage system availability and security posture

2.3 Design and Implementation Constraints

- Privacy and confidentiality: student identity and personal information must be protected; anonymized identifiers must be used in analysis outputs by default
- Analysis restriction: the system must implement its own comparison engine; direct file-to-file comparison cannot be outsourced to an external AI service
- Minimum language support: must support C, C++, and Java at minimum

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

- Repository model: must support comparisons within the current offering and against instructor-selected secondary repositories
- Web-based delivery: analysis review must be accessible through a browser-based UI
- Security constraints: role-based access control for instructor/TA/admin functions; audit logging for sensitive actions (especially identity reveal if supported)

2.4 Assumptions and Dependencies

Assumptions:

- Students will submit source code only in the supported languages (C, C++, Java) and in an instructor-approved packaging format (e.g., a single source file or a compressed archive containing a project).
- Submissions will comply with configured constraints such as maximum upload size, permitted file extensions, and directory structure rules for archived projects (if archives are allowed).
- The instructor will define which repositories are used for each analysis run (primary course repository and any selected secondary repositories) and will set review thresholds (e.g., "flag pairs above X% similarity").
- If common starter code or templates are distributed to the class, the instructor will provide the starter code files (or an identifier list) so the system can optionally exclude or down-weight these portions during similarity scoring to reduce false positives.
- Users (students and instructors) will access the system using a modern web browser and a stable internet connection; students are responsible for uploading correct and complete files.
- The system will operate under the assumption that user accounts and roles (student, instructor, administrator) are managed within the application and that users only access features permitted by their role.

Dependencies:

- Hosting infrastructure: the project depends on the availability of a server environment capable of running a web application backend, maintaining persistent storage for submission artifacts, and supporting background processing for analysis jobs.
- Persistent storage: the system depends on reliable storage for (a) raw uploaded submissions, (b) extracted/normalized intermediate representations used by the similarity engine, and (c) generated analysis reports. Storage must remain available across restarts and deployments.
- Database availability: the system depends on a relational database to store submission metadata, repository definitions, similarity scores, analysis run history, and audit logs.
- Language processing components: the system depends on a reliable method to tokenize/parse and normalize source code in C, C++, and Java. This may be implemented through custom parsing/tokenization logic or through approved third-party libraries; in either case, correctness of similarity scoring depends on these components.
- System time and logging: the system depends on accurate server time for timestamps (submission time, analysis time) and on logging facilities to support troubleshooting and auditability (especially for actions that could expose sensitive information).
- Security mechanisms: the system depends on correct implementation of authentication, role-based authorization, and access control to ensure confidentiality and to prevent unauthorized access to submissions or analysis results.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

3. System Features

3.1 User Authentication and Authorization

3.1.1.1 Description

Allows access to the system with a role-based design. High priority for security reasons.

3.1.1.2 Stimulus/Response Sequences

System will first prompt user to log in with a username and password. Once logged in, the system will display a log out button for all roles

Professor/Teaching Assistant
Logs in with username and password
Has access to all student submissions

Student
Logs in with username and password
Able to access their own submission
Able to create a new submission

Administrator
Logs in with username and password
Access to professor view and student view
Display performance metrics for debugging

3.1.1.3 Functional Requirements

1: Allow users to log in with valid credentials
2: Support roles; Professor/TA, Student, Administrator
3: Restrict access to functions based on user role
4: Allow authenticated users to log out and terminate their session

3.2 Course and Assignment Management

3.2.1.1 Description

Allows professors and administrators to create course and assignment contexts for students to create submissions

3.2.1.2 Stimulus/Response Sequences

Users with an admin/professor role will be prompted to create a course and create assignments within the course. There will be a drop down menu for professors to access multiple courses. Each course will display a list of assignments when clicked. Each course will show a list of submissions when clicked.

3.2.1.3 Functional Requirements

1: Allow professor/admin to create and manage classes
2: Allow professor/admin to create and manage assignments
3: Allow assignment configuration, supporting Java, C, and C++
4: Associate each assignment with a submission repository
5: Allow admin role to manage course rosters

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

3.3 Code Submission

3.3.1.1 Description

Allows students to submit their code to an assignment within a course.

3.3.1.2 Stimulus/Response Sequences

Students will see a box to upload a file by either dragging the file into the box or by clicking upload. After clicking upload, the system will open the students file manager and the student can choose what to upload. After the student confirms the file, the web interface will display the file name in the dropbox and show a submit button. When the student clicks submit, the dropbox disappears and the file name is displayed and the system sends a confirmation email to the student.

3.3.1.3 Functional Requirements

1: Allow students to upload code submissions to a web interface
2: Validate submissions to be Java, C, or C++ files based on assignment, display reason for rejection to the student
3: Store submissions in an assignment repository
4: Record submission data such as name, timestamp, language
5: Provide submission confirmation to the student by email

3.4 Repository Management

3.4.1.1 Description

All code submissions are held in a SQL database based on course and assignment. Professor has their own personal repository of code to use for comparisons with any assignment repository. Students will see a box to upload a file by either dragging the file into the box or by clicking upload. After clicking upload, the system will open the students file manager and the student can choose what to upload. After the student confirms the file, the web interface will display the file name in the dropbox and show a submit button. When the student clicks submit, the dropbox disappears and the file name is displayed and the system sends a confirmation email to the student.

3.4.1.2 Functional Requirements

1: The system will hold all code submissions in a repository organized by course and assignment
2: Allow professor to access student submissions by displaying the code
3: Allow professor to upload personal files to their own repository

3.5 Similarity Analysis

3.5.1.1 Description

Executes code similarity analysis across selected repositories

3.5.1.2 Stimulus/Response Sequences

Professor can select which repositories to check against each other, including their personal folder. After selecting and clicking the confirm button, the program displays a list of submissions with plagiarism percentages. The professor can select a submission to see a side-by-side comparison.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

3.5.1.3 Functional Requirements

1: Allow a professor/TA to initiate an analysis for an assignment
2: Perform analysis using an internally implemented comparison engine
3: Compute a plagiarism percentage for each compared submission
4: Store analysis results with respect to parameters
5: Support C, C++, and Java source code
6: Provide students with a plagiarism percentage for their own submission
7: Ensure students cannot access the plagiarism percentage from other submissions

4. Data Requirements

This section describes the data that the system consumes as inputs, processes internally, and produces as outputs during its operation. The proposed system manages academic code submissions and performs similarity analysis using an internally developed comparison engine. As a result, the system must store and process not only raw uploaded data, but also derived artifacts generated during preprocessing and analysis, while ensuring data integrity, traceability, and compliance with privacy requirements.

The primary categories of data handled by the system include:

- user and access control data,
- course, assignment, and submission metadata,
- repository data uploaded as ZIP archives,
- extracted and processed source code files,
- similarity analysis configurations and results,
- anonymization mappings and audit records.

Together, these data elements support the system's core functionality: accepting submissions, comparing them against selected repositories, producing similarity results, and presenting those results in a secure and auditable manner.

4.1 Logical Data Model

The logical data model represents the major data objects managed by the system and the relationships between them. It provides a conceptual view of how data is organized, independent of any specific database technology or implementation decisions. This model reflects both the academic workflow (courses, assignments, submissions) and the technical workflow (repository ingestion, analysis execution, result generation).

At the highest level, the system manages **users**, who interact with **courses** and **assignments** in different roles. Students submit work for assignments, while instructors and administrators upload repositories and initiate similarity analyses. Submissions and repositories are both treated as structured collections of files, with repositories specifically originating from ZIP archive uploads.

Each repository upload represents a versioned snapshot of a repository at a particular point in time. This allows the system to preserve historical context and ensures that similarity results remain reproducible even if a repository is updated or replaced later. Extracted files from both submissions and repositories are stored as individual logical entities so that comparisons can be performed at the file level rather than treating uploads as opaque blobs.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

Similarity analysis is modeled explicitly through analysis runs. An analysis run captures who initiated the analysis, which submissions and repositories were included, and which configuration parameters and engine version were used. The results of an analysis run consist of one or more similarity results, each representing a comparison between two source code artifacts. Where applicable, detailed match information may be associated with a result to support instructor review.

To support privacy requirements, anonymization mappings are associated with individual analysis runs. These mappings allow the system to present anonymized identifiers in reports and interfaces without permanently dissociating results from their original sources. Finally, audit log entries record significant actions performed within the system to support accountability and compliance.

This logical model is represented visually using an entity–relationship diagram (ERD), which illustrates entities such as User, Course, Assignment, Submission, Repository, AnalysisRun, and SimilarityResult, along with their relationships.

4.2 Data Dictionary

The data dictionary defines the structure and meaning of the data elements used by the system. It specifies the composition of data entities and documents the data type, format, constraints, and allowed values for each attribute. The data dictionary serves as a shared reference for developers, designers, and reviewers, ensuring consistent interpretation of system data.

For this project, the data dictionary is maintained as a separate artifact rather than being fully embedded within the SRS. This approach improves readability of the SRS and allows the data dictionary to be reused or expanded during later project phases. The dictionary includes, at minimum, definitions for identifiers, timestamps, hashes, configuration parameters, similarity scores, and status fields.

Special attention is given to fields that affect traceability and integrity, such as cryptographic file hashes, engine version identifiers, and analysis configuration records. Enumerated fields are used where appropriate to constrain valid values and reduce ambiguity, for example when recording file inclusion status, analysis run state, or user role.

4.3 Reports

The system generates several logical reports intended to support academic review, administrative oversight, and user confirmation. These reports are defined in terms of their content and purpose rather than fixed visual layouts, which are deferred to the design stage.

The primary report generated by the system is the **Similarity Summary Report**, which presents the results of a similarity analysis run. This report includes metadata describing the analysis configuration, the repositories used, and a ranked list of detected similarities, sorted by similarity score. The report is intended for instructor use and supports filtering and prioritization of results.

The system also produces a **Submission Receipt Report** for students, confirming that a submission was received and processed. This report includes submission timestamps and file counts but does not expose similarity results or repository data.

For administrative and instructional oversight, the system may generate a **Repository Ingestion Report**, summarizing the contents of an uploaded repository ZIP file. This report describes how many files were extracted, how many were excluded, and the reasons for exclusion.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

Where detailed review is required, a **Detailed Match Report** may be generated, describing specific comparisons between code artifacts. By default, this report presents anonymized identifiers to protect student privacy, with access to real identities restricted to authorized roles.

4.4 Data Acquisition, Integrity, Retention, and Disposal

4.4.1 Data Acquisition

Data is acquired by the system through controlled upload mechanisms provided by the web interface. Student submissions are uploaded in association with a specific assignment, while repositories are uploaded as ZIP archives by instructors or administrators. Upon upload, the system validates the received data, stores the original archive or file package, and extracts individual source code files for further processing.

During acquisition, the system records metadata such as upload timestamps, file sizes, and cryptographic hashes. This information is used to verify data integrity and to detect duplicate or altered uploads. Files that do not meet defined acceptance criteria, such as unsupported file types or excessive size, are excluded from further processing and recorded accordingly.

4.4.2 Data Integrity and Protection

Maintaining the integrity of stored data is a core requirement of the system. The system ensures that uploaded artifacts are not modified after ingestion by storing immutable copies of original ZIP archives and computing hashes for both archives and extracted files. Referential integrity between related data objects, such as analysis runs and their results, is enforced to prevent orphaned or inconsistent records.

To protect data against accidental loss or corruption, the system supports regular backups of both metadata and stored artifacts. Recovery mechanisms are designed to restore the system to a consistent state, ensuring that original uploads, extracted data, and analysis results remain aligned.

4.4.3 Data Retention

The system enforces defined data retention policies that balance academic needs, legal compliance, and storage constraints. Different categories of data are retained for different durations. For example, student submissions and associated analysis results are retained for the duration of the course offering and any applicable appeal period, while repository data from past offerings may be retained for longer periods to support longitudinal comparisons.

Retention policies are configurable by administrators to allow alignment with institutional or departmental guidelines. Temporary and intermediate data generated during analysis, such as cached artifacts or transient processing files, are retained only as long as necessary to complete analysis tasks.

4.4.4 Data Disposal

When data reaches the end of its retention period or is explicitly deleted by an authorized user, the system performs controlled disposal procedures. Disposal includes removal of primary data (such as ZIP archives and extracted files), derived data (such as normalized representations or similarity artifacts), and associated metadata.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

The system ensures that disposal actions do not leave residual data accessible through caches, temporary storage, or outdated backups. Where required, anonymized summaries may be retained for statistical or audit purposes without preserving identifiable student data.

4.4.5 Privacy and Anonymization

To comply with privacy requirements, the system separates identity information from similarity results. Anonymized identifiers are used by default in reports and user interfaces, with real identities accessible only to authorized roles. Anonymization mappings are scoped to individual analysis runs to prevent cross-run correlation of identities.

Access to sensitive data is governed by role-based controls, ensuring that users can view only the information necessary for their role.

5. External Interface Requirements

This section explains the system's interactions with users as well as external software. It defines the necessary interfaces to guarantee proper communication between the system and its environment.

5.1 User Interfaces

The system must provide a web-based user interface that is accessible via standard web browsers. All interfaces must be role-based, ensuring that users are only provided with functionality suitable for their assigned role. Navigation must be straightforward and consistent throughout the system, and the interface must provide relevant feedback for successful activities, warnings, and error conditions.

Students will interact with the system via an interface that allows them to authenticate, view enrolled courses and assignments, upload programming submissions, and receive confirmation of submission. Students may access the similarity score connected with their own submission, but they are not permitted to view peer names, peer submissions, or detailed similarity analysis results.

Instructors and Teaching Assistants must be given interfaces that allow them to administer courses and assignments, create repositories, and initiate similarity analysis. The system will provide similarity analysis results on a dashboard that includes prioritized similarity scores along with relevant metadata. Authorized instructional staff members must additionally have access to a side-by-side code comparison view that shows observed similarities in order to facilitate human inspection.

Administrators must interact with the system via interfaces for user and role management, repository administration, audit log inspection, and system configuration. Administrative interfaces must be restricted to authorized users only.

5.2 Software Interfaces

To provide functionality, the system will interface with a number of internal software components. A web browser will serve as the client interface for all user interactions. Client requests are processed by an application server, which is in charge of authentication, business logic, and system operation coordination.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

The application server must connect with a similarity analysis engine, which compares code and returns similarity results. A database management system is used to store and retrieve structured data such as user information, courses, assignments, analysis of metadata, and audit records. Uploaded source code files and repository artifacts must be stored on a server. The data shared between these components must be checked to ensure integrity and consistency.

5.3 Communications Interfaces

The system must use common network communication protocols to facilitate interaction between client devices and server components. To ensure data security during transmission, communication between the web client and the application server must take place over secure HTTPS connections. Internal communication between server-side components shall take place via the hosting environment's internal network or communication mechanisms. The system must allow several users to connect to the network at the same time without requiring additional messaging or communication services.

6. Quality Attributes

6.1 Usability

- The system shall be usable on modern desktop browsers (Chrome, Firefox, Edge, Safari) without requiring plugins
- After upload, the system shall display a confirmation screen including submission timestamp and a submission ID
- The instructor dashboard shall allow filtering results by assignment, date range, and similarity threshold
- The comparison viewer shall present two submissions side-by-side with highlighted matching regions and line numbers
- All error messages shall be actionable (what happened + how to fix), and must not reveal private data

6.2 Performance

- The system shall accept uploads up to 20 MB per submission and respond with an acknowledgement in ≤ 5 seconds under normal load
- The system shall allow the instructor to open any generated report in ≤3 seconds once analysis is complete
- The similarity engine shall process supported languages (C/C++/Java minimum) and produce a similarity score for each pair without crashing on malformed input
- Long-running analysis shall run asynchronously and expose progress/status (Queued/Running/Complete/Failed)

6.3 Security

- The system shall enforce authentication for all instructor and admin features
- A student shall only be able to view their own submission status/receipt, not other students' submissions

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

- Submitted files shall be stored with access controls such that only authorized roles can retrieve them
- The system shall keep an audit log of instructor actions (login, report views, comparisons viewed, repository selections)
- The system shall validate uploaded files to reduce risk from malicious uploads (type/extension checks + size limits)

6.4 Privacy/Confidentiality

- The system shall anonymize submissions before displaying results to external clients, exposing only identifiers
- The instructor view shall support re-identification only for authorized instructors of the course/assignments
- Reports exported or shared shall exclude student personal identifiers by default
- Logs shall not store raw code content; they shall store references (submission IDs) and metadata only

6.5 Reliability/Availability

- The system shall not lose a successfully acknowledged submission; submissions must be durable (stored before confirmation)
- If analysis fails for a submission pair, the system shall mark that pair as failed and continue processing other pairs
- The system shall perform automatic backups of metadata (DB) at a defined interval
- The system shall provide clear recovery guidance to admins for restoring from backups

6.6 Maintainability/Modifiability

- The system shall separate the comparison engine, repository/storage layer, and web UI into distinct modules/services
- Adding a new language support shall require implementing a new “normalizer/tokenizer” module without rewriting the whole engine
- The system shall include developer documentation for setup, configuration, and running tests

6.7 Portability/Deployment

- The system shall be deployable on a single server with documented prerequisites
- The system shall provide a one-command startup option or a clearly documented manual deployment procedure

7. Glossary

- UUID – Universal Unique Identifier
- Plagiarism Percentage – Percentage score that represents how similar a submission is to another

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

8. Host Software Requirements and Viability

8.1 Purpose

The software, platform, and hosting environment needed to install and run the Code Submission and Similarity Detection System are described in this section. These specifications specify the system's operating environment and show that the suggested solution is achievable with widely accessible and stable technologies.

8.2 Client Environment Requirements

- A current web browser must be able to access the system.
- Current versions of the following browsers will be supported:
    - Google Chrome
    - Mozilla Firefox
    - Apple Safari
    - Microsoft Edge
- A regular web browser is the only client-side software that needs to be installed.

Justification:
For students and teaching staff, a browser-based client guarantees platform neutrality and wide usability. This strategy reduces deployment and maintenance costs and meets with the project statement's demand that the customer be reachable online.

8.3 Server Operating System Requirements

- A Multipurpose server OS must be able to install the system.
- Linux-based server versions, such as Ubuntu Server, and Microsoft Windows Server belong to the OS (operating system) that are supported.

Justification:
The system is independent of features unique to the operating system. Portability and compatibility with standard academic hosting settings are ensured by limiting requirements to widely used server operating systems.

8.4 Application Runtime Environment

- The server environment shall support a backend application runtime capable of:
    - hosting a web-based application,
    - manage file uploads and downloads,
    - operate the internally developed/implemented similarity comparison engine.
- The runtime environment may be implemented using any modern, supported application platform suitable for server-side processing.

Justification:
No particular programming language or framework is required per the project requirements. System

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

functionality can be supported by any developed runtime that can handle file processing, database interfaces, and mathematical operations.

8.5 Requirements for Database Management System (DBMS)

- A relational database management system (RDBMS) will be used by the system.
- Acceptable database systems include:
  - PostgreSQL,
  - MySQL,
  - SQLite (for smaller-scale or development deployments)

Database Responsibilities:

- User accounts and role information
- Course and assignment metadata
- Submission metadata (without raw code files)
- Similarity analysis results
- Audit and system logs

Justification:
Relational databases offer efficient querying, referential integrity, and organised data storage that are suitable for handling system information and analysing results.

8.6 File Storage Requirements

- The system must give the application server access to continuous and permanent file storage.
- File storage may be implemented using:
  - a server-based file system
  - managed storage available through the host operating system.

Stored Data shall Include:

- Student-submitted source code files and directories
- Repository data used for similarity comparison
- Necessary Files required for side-by-side comparison views

Justification:
While maintaining original submissions, keeping metadata and artefact storage apart increases maintainability, reliability and auditability.

8.7 Network and Communication Requirements

- The system must be reachable via the public internet or a private network.
- Secure communication methods (e.g., HTTPS) are recommended to protect:
  - authentication credentials,

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

  - submitted code,
  - similarity analysis results.
- Throughout submission, analysis, and result display, the system relies on continuous connection to the network between the client and server.

Justification:
In order to support web-based clients and adhere with confidentiality and privacy regulations related to the transfer of student data, network-based access is mandatory.

8.8 Hardware Requirements

- No specific hardware is needed to run or host the system.
- The system shall be capable of running on standard server hardware with:
  - a general-purpose CPU,
  - sufficient memory to support multiple users and concurrent submission and analysis operations,
  - Sufficient disk storage to store submissions and repository data.

Justification:
The similarity comparison engine does not require hardware accelerators like GPUs because it is built as a software-based feature.

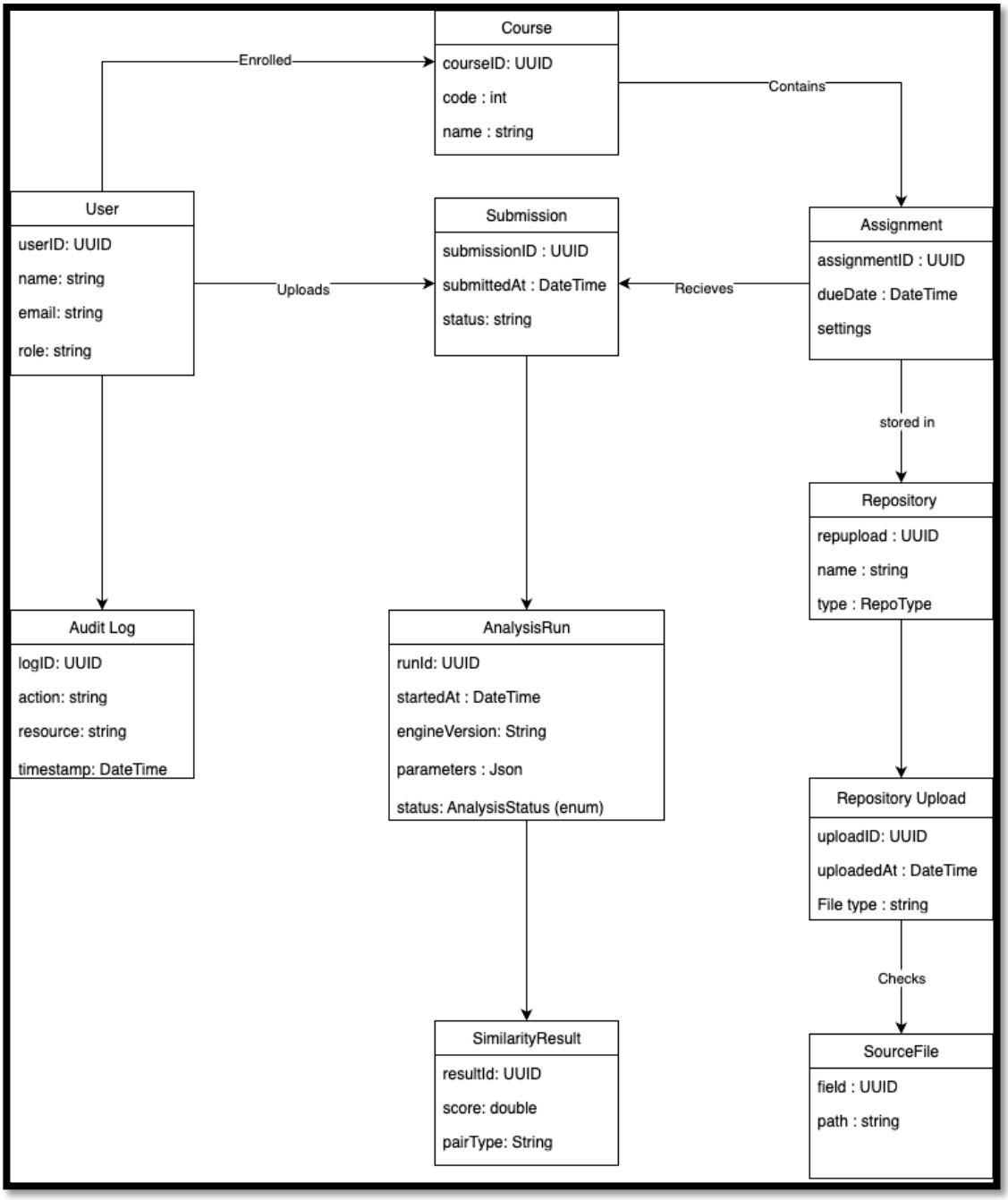
8.9 Viability and Feasibility Assessment

Within the limits of the project, the host software requirements specified in this section are achievable and reasonable. Modern web browsers, common server operating systems, backend runtime environments, relational databases, and file storage systems are all widely used, readily available, and fully explained.

The system architecture is independent of particular system components, additional AI services, or unique platforms. Because of this, the system can be set up on a single server for small-scale use or, if necessary, expanded to larger environments. The practicality of the suggested hosting environment is confirmed by the regular operation of other web-based academic systems under similar circumstances.

9. Analysis Models

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.



Model 1.1 Class Diagram

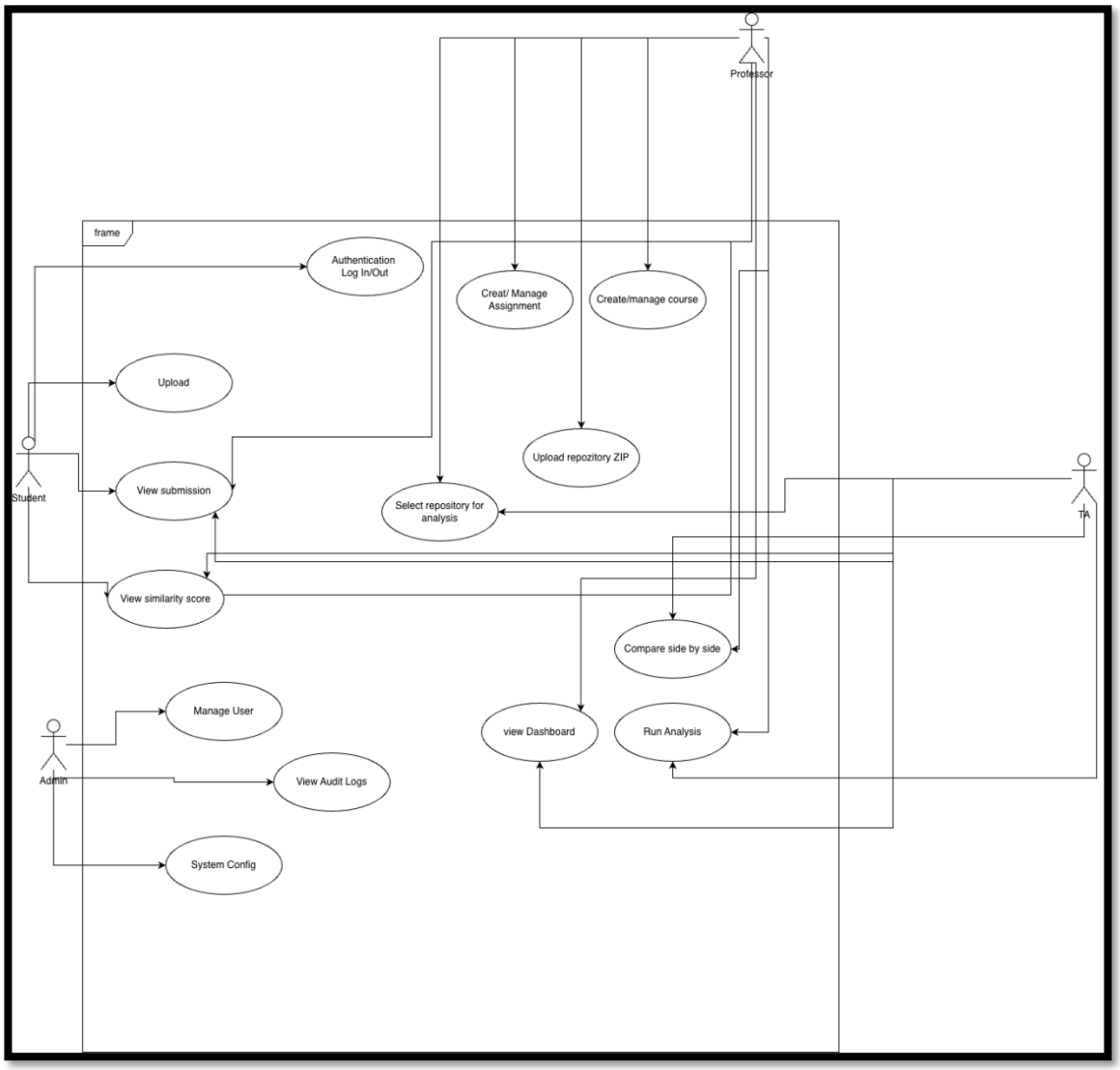
The class diagram represents the core data structure of the Similarity Analysis Platform and defines how users, courses, submissions, repositories, and analysis results are related.

A User is enrolled in a Course and can upload one or more Submissions. Each submission records metadata such as submission time and status and belongs to a specific Assignment. Assignments are contained within courses and define configuration settings such as due dates.

Assignments are associated with one or more Repositories, which store reference code used for similarity analysis. Repositories are uploaded as versioned RepositoryUpload instances, each of which is extracted into individual SourceFile objects.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

Each Submission can trigger an AnalysisRun, which captures execution details such as engine version, parameters, start time, and analysis status. An analysis run produces one or more SimilarityResult objects, each storing similarity scores and comparison type.
System actions performed by users are recorded in the AuditLog, enabling traceability and administrative oversight.



Model 1.2 Use Case

The use case diagram models a Similarity Analysis Platform that supports assignment submission and plagiarism analysis in an academic environment. All system functionality is contained within a single system boundary, with users interacting from outside the system.
The platform supports four actors: Student, Professor, Teaching Assistant (TA), and Administrator, each with clearly defined responsibilities.

Students can upload submissions, view submission status, and view their similarity scores after analysis is completed.

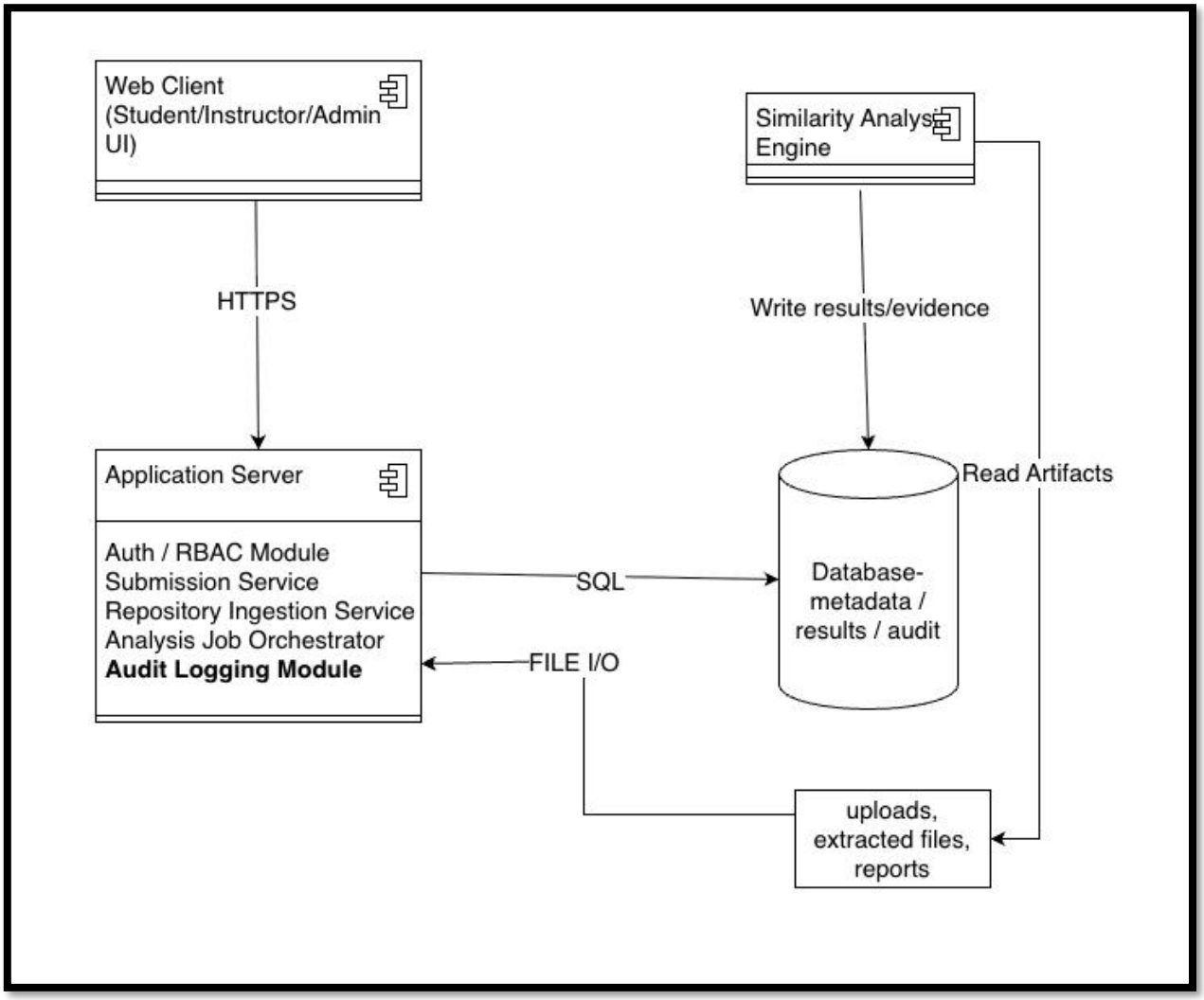
Professors can create and manage courses and assignments, upload and select reference repositories, run similarity analysis, view analytical dashboards, and compare submissions side-by-side.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.

Teaching Assistants support the analysis workflow by managing assignments, uploading and selecting repositories, running analysis, viewing dashboards, and comparing submissions, but do not manage courses.

Administrators handle system-level operations, including user management, audit log inspection, and system configuration.

All protected actions require user authentication (log in/log out).



Model 1.3 Component Diagram

A UML component diagram of the system architecture: A web-based client connects to the application server via HTTPS as part of the system's client-server architecture. Important functions including as role-based access control and authentication, submission processing, repository entry, analysis task management, results reporting, and audit logging are all contained within the application server. Computational similarity analysis is carried out using a specialized similarity analysis engine that communicates with permanent storage for results and artifacts. While raw uploads, extracted source files, and generated reports are kept in file storage through file input/output operations, organized file metadata, similarity results, and audit records are kept in a database that can be accessed using SQL. Maintainability, scalability, and secure processing are all improved by the separation of responsibilities.

Copyright © 2023 by Karl Wiegers and Seilevel Partners LP. Permission is granted to use and modify.